



UNIVERSIDAD CENTRAL DE VENEZUELA
FACULTAD DE CIENCIAS
ESCUELA DE COMPUTACIÓN
ALGORITMOS Y ESTRUCTURA DE DATOS
SECCIÓN C1

INFORME PROYECTO#1

Auxiliar Docente:

Brayan Silva

Erimar Reis

Alumnos:

José Daniel Andrade C.I:29617497

Carlos Trujillo C.I:31341616

Caracas, 10 de diciembre del 2025

INFORME

Reporte de entrega del sistema "Vista de Cazador" (Proyecto X78A). El equipo Delta informa que se completó con éxito el sistema solicitado para detectar y elegir objetivos en los autómatas X78A-Cazador. A continuación, se explican las partes técnicas del sistema, centrándose en los tres puntos clave: Lectura y Guardado, Detección por Relleno y Selección Mejorada.

LECTURA Y GUARDADO DE DATOS

Para manejar la imagen y los objetivos, se creó un struct llamado Circulo que guarda los datos de cada objetivo encontrado. Esta estructura almacena el color, los bordes y las medidas calculadas.

```
struct Circulo {  
    int color;  
  
    int minX, maxX;  
    int minY, maxY;  
  
    int centroX, centroY;  
    int diametro;  
    int radio;  
    int circunferencia;  
  
    double pi_local;  
    int area_normalizada;  
};
```

La lectura del archivo PGM se hizo en la función cargarImagen. Se usó la librería fstream para leer el encabezado, obteniendo el tamaño (ancho, alto) y el color máximo (max_gris). Después, se usaron dos ciclos para guardar los píxeles en la matriz global imagen[MAX_ALTO][MAX_ANCHO], lo que permitió usar los datos rápidamente.

DETECCIÓN DE OBJETIVOS

Para encontrar los objetivos no se usó una revisión simple, sino un algoritmo de Búsqueda en Profundidad adaptado para zonas conectadas, creado en la función Ver_vecinos.

El proceso principal, procesarImagen, revisa la matriz buscando píxeles que no sean fondo y no se hayan visitado antes. Al encontrar uno, se inicia la función recursiva:

```

void Ver_vecinos(int x, int y, int colorBuscado, int i) {

    if (x < 0 || x >= ancho || y < 0 || y >= alto) return;

    if (visitado[y][x]) return;
    if (imagen[y][x] != colorBuscado) return;

    visitado[y][x] = true;

    if (x < lista[i].minX) lista[i].minX = x;
    if (x > lista[i].maxX) lista[i].maxX = x;
    if (y < lista[i].minY) lista[i].minY = y;
    if (y > lista[i].maxY) lista[i].maxY = y;

    Ver_vecinos(x + 1, y, colorBuscado, i);
    Ver_vecinos(x - 1, y, colorBuscado, i);
    Ver_vecinos(x, y + 1, colorBuscado, i);
    Ver_vecinos(x, y - 1, colorBuscado, i);
}

```

Funcionamiento:

Esta función extiende la búsqueda desde un píxel inicial hacia sus 4 vecinos. Mientras avanza, actualiza los valores mínimos y máximos de X e Y en la lista del círculo i. Esto permitió marcar el recuadro que encierra al círculo sin tener que revisar la imagen muchas veces.

CÁLCULO DE DATOS GEOMÉTRICOS

Una vez separados los objetivos, la función calcularDatos pasó a ajustar la información.

- **Bordes:** Para obtener la circunferencia, se recorrió el área dentro del recuadro del círculo. Se contó como "borde" cualquier píxel del color del círculo que tuviera al menos un vecino de otro color o tocara el final de la imagen.
- **Área Normalizada:** Primero se sacó el promedio de los pi locales (circunferencia /diámetro) de todos los objetivos. Luego, se usó la fórmula pedida: $\pi \times \text{radio al cuadrado}$.

// Parte del cálculo de Pi local

```

if (circuloActual.diametro > 0)
    circuloActual.pi_local = (double)circuloActual.circunferencia / (double)circuloActual.diametro;
else
    circuloActual.pi_local = 0;

sumaTotalPi = sumaTotalPi+ circuloActual.pi_local;

```

SELECCIÓN DE OBJETIVOS (BACKTRACKING)

La parte más importante fue elegir los objetivos para aprovechar al máximo los disparos disponibles (T). Para esto, se creó la función `buscarMejorSuma`.

Este método de backtracking prueba todas las opciones: para cada círculo, decide si "sumarlo" o "no sumarlo".

```
void buscarMejorSuma(int i, int sumaActual, bool seleccionActual[]) {  
    if (sumaActual > T) return;  
  
    int diferencia = T - sumaActual;  
    if (diferencia < menorDiferencia) {  
        menorDiferencia = diferencia;  
        sumaMaxima = sumaActual;  
  
        for (int k = 0; k < M; k++) {  
            mejorSeleccion[k] = seleccionActual[k];  
        }  
    }  
  
    if (i == M) return;  
  
    seleccionActual[i] = true;  
    buscarMejorSuma(i + 1, sumaActual + lista[i].area_normalizada, seleccionActual);  
  
    seleccionActual[i] = false;  
    buscarMejorSuma(i + 1, sumaActual, seleccionActual);  
}
```

Lógica del Algoritmo:

1. Corte por exceso: Si `sumaActual` es mayor a `T`, la función regresa de inmediato, evitando cálculos que no sirven.
2. Guardar el mejor: En cada paso válido, se revisa si la diferencia (`T - sumaActual`) es menor que la `menorDiferencia` encontrada hasta ahora. Si es así, se actualiza la `mejorSeleccion`.
3. Prueba completa: Se asegura encontrar la mejor combinación probando las dos opciones (tomar o dejar) para cada uno de los `M` círculos.

GENERACIÓN DE SALIDA

Finalmente, la función ArchivoSalida creó una imagen PGM ("sites.pgm") para ver los resultados. Se recorrió cada píxel y se revisó si era parte del borde o el centro de alguno de los círculos, pintándolos con su color o con blanco (max_gris) para el centro.

CONCLUSION

El sistema desarrollado cumplió satisfactoriamente con la detección y caracterización de objetivos circulares discretos. La implementación de algoritmos recursivos para la detección de regiones y la combinatoria de sumas permitió resolver el problema de forma eficiente dentro de las limitaciones impuestas, garantizando la precisión en los cálculos geométricos normalizados y la correcta generación de los reportes visuales y textuales solicitados.